

Open-Source Contribution Tracker and Recommendation System for Beginner Developers

A CASE STUDY ANALYSIS REPORT

Submitted in the partial fulfilment for the Course of

CSA– SOFTWARE ENGINEERING

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE

Submitted by

N.Goutham(192472024)

Under the Supervision of

Dr.ANITA DAVAMANI K

Chennai-602105

July 2026



SIMATS ENGINEERING
**Saveetha Institute of Medical and Technical
Sciences**



Table of Contents

S.No	Content	Page No
1	Understanding the Problem Statement 1.1 Introduction 1.2 Problem Statement in Own Words 1.3 Objectives 1.4 Scope	3-4
2	Stakeholder and Challenge Analysis 2.1 Stakeholders 2.2 Roles and Responsibilities 2.3 Challenges	5
3	Current System Analysis 3.1 Existing System 3.2 Strengths 3.3 Limitations 3.4 Problems and Gaps	6-7
4	Proposed Software Solution	8-10
5	SDLC / Framework Justification	11
6	Expected Outcomes and Limitations	12
7	Conclusion	13

1. Understanding the Problem Statement

Introduction

Open-source software has become an essential part of today's technology industry, with developers from around the world collaborating to build applications, frameworks, operating systems, and software libraries. Platforms such as GitHub provide access to millions of repositories where developers can contribute by fixing bugs, improving documentation, adding new features, and enhancing existing code. For students and beginner developers, contributing to open-source projects is one of the best ways to gain practical programming experience, strengthen problem-solving skills, and build a professional portfolio. However, the vast number of available repositories and the varying complexity of projects make it difficult for beginners to identify suitable opportunities. As a result, many aspiring contributors struggle to take their first step into the open-source community. The proposed system aims to simplify this journey by providing personalized project recommendations, contribution tracking, and progress monitoring through a single platform.

Problem in Own Words

Many beginner developers are interested in contributing to open-source projects because it helps them improve their technical skills and increases their chances of securing internships and job opportunities. However, when they visit GitHub, they are often overwhelmed by the enormous number of repositories and issues available. Most projects are designed for experienced developers, making it difficult for beginners to understand the project structure, contribution guidelines, and coding standards. Even though GitHub provides labels such as "Good First Issue" and "Help Wanted," finding issues that match a developer's programming skills and interests still requires considerable manual effort.

Another major challenge is the lack of a centralized platform to monitor contribution progress across multiple repositories. Beginners often find it difficult to keep track of the pull requests they have submitted, the issues they have resolved, and the technologies they have worked with over time. This makes it harder for them to measure their learning progress or showcase their achievements during placement or internship opportunities. Therefore, there is a need for a software solution that not only recommends suitable beginner-friendly projects but also tracks contributions, visualizes skill development, and generates a professional portfolio that reflects the developer's open-source journey.

Objectives of the Proposed System

The primary objectives of the proposed system are:

- To recommend beginner-friendly open-source repositories and issues based on the user's programming languages, technical skills, and areas of interest.
- To integrate with the GitHub API and automatically retrieve repositories containing labels such as "Good First Issue" and "Help Wanted."

- To provide a centralized contribution tracking system that records pull requests, issue activities, merge status, and overall contribution history across multiple repositories.
- To visualize the user's skill progression through an interactive dashboard displaying contribution frequency, technologies used, repository diversity, and overall learning growth.
- To generate a professional open-source portfolio that summarizes the user's contributions and can be included in resumes, LinkedIn profiles, or job applications.
- To improve beginner engagement in open-source communities by providing personalized recommendations, timely notifications, and insights into mentor-friendly and responsive projects.

Scope of the Proposed System

The scope of the proposed system is focused on supporting beginner developers who want to start contributing to open-source projects. The application provides personalized repository recommendations, tracks contribution activities through GitHub integration, generates visual dashboards to monitor learning progress, and creates professional portfolios highlighting the user's achievements. It also offers notifications for newly available beginner-friendly issues and community insights that help users identify active and supportive projects. The initial implementation is limited to repositories hosted on GitHub and is primarily designed for entry-level contributors. Future versions of the system can be extended to support additional platforms such as GitLab and Bitbucket, incorporate artificial intelligence for more accurate recommendations, and introduce advanced analytics for experienced developers.

2. Stakeholder and Challenge Analysis

Stakeholders

Beginner developers, project maintainers, mentors, recruiters, administrators and GitHub. From a software engineering perspective, the proposed solution follows modular architecture, layered design and API-based integration. Each module performs a specific responsibility, improving maintainability, testing and scalability. Security mechanisms, proper documentation, validation, logging and exception handling improve reliability, while an intuitive interface ensures usability for beginner developers. Continuous enhancement through Agile development allows the application to evolve with user feedback and future requirements.

Roles and Responsibilities

Beginner Developers: Create profiles, contribute to beginner-friendly projects, and track their learning progress through the platform.

- **Open-Source Project Maintainers:** Manage repositories, review pull requests, label beginner-friendly issues, and maintain project quality.
- **Mentors and Experienced Contributors:** Guide beginners by providing technical support, reviewing code, and sharing best practices.
- **Recruiters and Hiring Managers:** Evaluate developers' portfolios and contribution history to assess their practical skills for internships and jobs.
- **System Administrator:** Manage users, maintain system security, monitor application performance, and ensure smooth platform operation.
- **GitHub Platform (API Provider):** Provide repository, issue, pull request, and user contribution data through secure GitHub APIs.

Challenges

Repository overload, lack of personalized recommendations, inconsistent documentation, and difficulty tracking progress. From a software engineering perspective, the proposed solution follows modular architecture, layered design and API-based integration. Each module performs a specific responsibility, improving maintainability, testing and scalability. Security mechanisms, proper documentation, validation, logging and exception handling improve reliability, while an intuitive interface ensures usability for beginner developers. Continuous enhancement through Agile development allows the application to evolve with user feedback and future requirements.

3. Current System Analysis

Existing System

The current system mainly relies on GitHub, where developers manually search for open-source repositories based on their interests and programming languages. Beginners use search filters, repository tags, and labels such as "Good First Issue" or "Help Wanted" to find suitable contribution opportunities. After selecting a repository, they read the documentation, understand the project structure, choose an issue, make code changes, and submit a pull request for review. Although GitHub provides all the necessary tools for open-source collaboration, beginners must perform most of these tasks manually without personalized guidance or recommendations.

Strengths of the Existing System

The existing system offers several advantages that support collaborative software development:

- Provides access to millions of open-source repositories.
- Supports version control using Git for efficient code management.
- Enables collaboration through pull requests, issues, and discussions.
- Offers labels such as "Good First Issue" and "Help Wanted" to identify beginner-friendly tasks.
- Maintains complete contribution history for every developer.
- Provides REST and GraphQL APIs for third-party integration.
- Supports code review and collaboration among developers worldwide.
- Includes project documentation and contribution guidelines for many repositories.

Limitations of the Existing System

Despite its advantages, the current system has several limitations from a beginner's perspective.

- No personalized recommendations based on programming skills.
- Beginners spend considerable time searching for suitable repositories.
- Difficulty levels of projects are not clearly categorized.
- No centralized dashboard to track contributions across multiple repositories.
- Lack of automatic portfolio generation for internships or placements.
- Limited guidance regarding mentorship and community support.

Problems and Gaps Identified

Although GitHub provides an excellent platform for software collaboration, it does not focus on the specific needs of beginner developers. New contributors often struggle to identify projects that match their knowledge and confidence level, resulting in frustration and reduced

participation. The absence of personalized recommendations and centralized progress tracking makes it difficult for beginners to measure their learning and showcase their achievements.

Another major gap is the lack of a system that combines project discovery, contribution tracking, skill visualization, and portfolio generation in a single application. As a result, users rely on multiple tools to manage their open-source journey. These limitations create the need for a dedicated platform that simplifies the contribution process and provides continuous guidance to beginner developers.

4. Proposed Software Solution

Overview

The proposed Open-Source Contribution Tracker and Recommendation System is a web-based application designed to simplify the open-source contribution process for beginner developers. The system integrates with the GitHub API to collect information about repositories, issues, pull requests, and contribution history. Based on the user's programming languages, technical skills, and areas of interest, it recommends suitable beginner-friendly repositories and issues. This personalized approach reduces the time spent searching for projects and helps users start contributing with confidence.

In addition to recommending projects, the system acts as a centralized platform for monitoring a developer's open-source journey. It tracks contribution activities, displays skill progression through an interactive dashboard, generates a professional portfolio, and notifies users when new beginner-friendly issues become available. By combining recommendation, tracking, analytics, and portfolio generation into a single application, the proposed solution provides a structured and user-friendly environment that encourages continuous learning and active participation in open-source communities.

Major Functional Modules

The proposed system consists of the following major functional modules:

- **User Authentication Module** – Allows users to securely register and log in using GitHub OAuth.
- **User Profile Management Module** – Stores programming languages, skills, interests, and experience level.
- **Recommendation Engine** – Suggests beginner-friendly repositories and issues based on the user's profile.
- **GitHub API Integration Module** – Retrieves repositories, issues, pull requests, and contribution details from GitHub.
- **Contribution Tracker Module** – Monitors pull requests, issues, merge status, and repository contributions.
- **Skill Progress Dashboard** – Displays contribution statistics, technology usage, and learning progress.
- **Notification Module** – Alerts users about newly available beginner-friendly issues matching their interests.
- **Portfolio Generation Module** – Creates a professional summary of the user's open-source contributions.
- **Admin Module** – Manages users, monitors system activities, and maintains application performance.

Functional Requirements

The system should be able to:

- Allow users to register and log in securely.
- Enable GitHub account integration using OAuth.
- Store and update user skills, interests, and experience level.
- Fetch repository and issue details using the GitHub API.
- Recommend repositories based on programming languages and interests.
- Display beginner-friendly issues such as "Good First Issue" and "Help Wanted."
- Track pull requests, issues, and merge status.
- Generate a contribution dashboard with analytics.
- Send notifications for newly available matching issues.
- Generate a downloadable open-source portfolio.
- Allow administrators to manage users and monitor system performance.

Non-Functional Requirements

The proposed system should satisfy the following non-functional requirements:

- **Performance:** The system should provide recommendations and load dashboards quickly.
- **Scalability:** It should support a growing number of users and repositories.
- **Reliability:** The application should provide accurate and consistent recommendation results.
- **Availability:** The platform should remain accessible with minimal downtime.
- **Security:** User authentication and data transmission should be secured using GitHub OAuth and HTTPS.
- **Usability:** The interface should be simple and easy for beginners to understand.
- **Maintainability:** The modular design should allow easy updates and feature additions.

Software Engineering Principles Applied

The proposed system is designed by following important software engineering principles to ensure quality, flexibility, and long-term maintainability.

Modularity: The application is divided into independent modules such as authentication, recommendation engine, contribution tracker, dashboard, notifications, and portfolio generation. Each module performs a specific function, making the system easier to develop, test, and maintain.

Scalability: The system is designed to accommodate an increasing number of users, repositories, and GitHub API requests without affecting performance. New features and modules can also be added in future versions.

Maintainability: A layered architecture and well-organized code structure make it easier to identify bugs, implement updates, and enhance existing functionalities without affecting other modules.

Reliability: The application ensures reliable operation by validating user inputs, handling API failures gracefully, maintaining accurate contribution records, and performing proper error handling.

Usability: The user interface is designed specifically for beginners, with simple navigation, clear recommendations, easy-to-understand dashboards, and intuitive workflows that reduce the learning curve.

Security: Security is maintained through GitHub OAuth authentication, encrypted communication using HTTPS, secure storage of user information, and proper access control to protect sensitive data.

5. SDLC / Framework Justification

Selected Methodology – Agile Scrum

The proposed system follows the Agile Scrum methodology because it is flexible and supports continuous development. Since the application depends on GitHub APIs, user feedback, and changing project requirements, Agile allows the development team to make improvements throughout the project instead of waiting until the end. The system can be developed in small, manageable iterations called sprints, where each sprint focuses on completing a specific set of features such as user authentication, recommendation engine, contribution tracker, or dashboard. This approach helps deliver a working product quickly while allowing new features to be added based on user needs.

Justification

Agile Scrum is the most suitable methodology for this project because the requirements may change as the system evolves. For example, new GitHub features, additional recommendation techniques, or enhanced dashboard functionalities can be incorporated without affecting the overall development process. Agile encourages regular communication among developers, testers, and stakeholders, making it easier to identify issues early and improve the quality of the application. It also reduces development risks by testing every module after each sprint instead of waiting until the entire project is completed.

Reasons for choosing Agile Scrum:

- Supports incremental and iterative development.
- Easily adapts to changing requirements.
- Encourages continuous testing and quality improvement.
- Delivers working software at the end of every sprint.
- Promotes teamwork and regular stakeholder feedback.
- Reduces project risks through early issue identification.

Development Process

The development of the proposed system follows a series of iterative sprints, where each sprint focuses on designing, developing, testing, and reviewing a specific module. Initially, user requirements are gathered and analyzed to understand the features required in the application. Based on these requirements, the development team plans the sprint activities and begins implementing individual modules such as authentication, GitHub integration, recommendation engine, contribution tracking, and dashboard generation. After implementation, each module undergoes testing to ensure it functions correctly before being integrated with the rest of the system.

6. Expected Outcomes and Limitations

Expected Outcomes

The proposed Open-Source Contribution Tracker and Recommendation System is expected to make the process of contributing to open-source projects simpler and more accessible for beginner developers. By providing personalized repository recommendations, centralized contribution tracking, and progress visualization, the system will help users improve their technical skills while reducing the time spent searching for suitable projects. It also encourages continuous learning by helping developers build a strong open-source profile that can support internship and job opportunities.

- Simplifies the process of finding beginner-friendly open-source projects.
- Recommends repositories based on user skills, interests, and programming languages.
- Reduces the time required to search for suitable contribution opportunities.
- Tracks pull requests, issues, and contribution history in one place.
- Displays learning progress through an interactive dashboard.
- Generates a professional portfolio of open-source contributions.
- Encourages beginners to participate actively in open-source communities.
- Improves collaboration between beginners, mentors, and project maintainers.
- Enhances practical programming and software development skills.
- Provides a scalable platform that can be expanded with additional features in the future.

Limitations

Although the proposed system provides several advantages, it also has certain limitations. Some functionalities depend on the availability and accuracy of data provided by the GitHub API. Repository recommendations may not always perfectly match a user's skill level because they are based on the available repository information and user profile. In addition, some advanced open-source projects may still require manual exploration and guidance from experienced contributors.

- Depends on the availability and response of the GitHub API.
- Recommendation accuracy depends on the quality of user profile information.
- Cannot guarantee that every recommended issue is suitable for every beginner.
- Some repositories may have incomplete or outdated documentation.
- Limited to GitHub repositories in the current version.
- Requires an active internet connection for real-time updates.
- Performance may be affected if GitHub API rate limits are exceeded.
- Does not automatically evaluate the quality of users' code contributions.
- Advanced project understanding still requires self-learning and mentor support.
- Additional integrations (such as GitLab or Bitbucket) are not included in the current implementation.

Future Enhancements

The proposed Open-Source Contribution Tracker and Recommendation System can be further enhanced by incorporating advanced technologies and additional features to improve the overall user experience. Future versions of the system can include AI and machine learning techniques to provide more accurate and personalized repository recommendations based on users' skills, interests, and contribution history. Support for other version control platforms such as GitLab and Bitbucket can expand the system beyond GitHub. Features like mentor matching, real-time notifications, gamification through badges and achievements, multilingual support, and mobile application development can further encourage beginner participation. The system can also integrate with professional networking platforms such as LinkedIn to automatically showcase users' open-source achievements. These enhancements will make the platform more intelligent, scalable, and beneficial for developers throughout their learning and professional journey.

Conclusion

This case study analyzed the challenges faced by beginner developers when contributing to open-source projects through GitHub. The existing system was studied to identify its strengths and limitations, and the need for a dedicated recommendation and contribution tracking platform was established. Based on this analysis, a proposed software solution was designed that integrates GitHub APIs to recommend beginner-friendly repositories, track contributions, monitor learning progress, and generate a professional portfolio.

From a software engineering perspective, the proposed system was planned using Agile Scrum methodology, modular architecture, and well-defined functional and non-functional requirements. The major system modules, stakeholder analysis, software engineering principles, expected outcomes, and project limitations were also identified. Overall, this study demonstrates that the proposed solution can simplify the open-source contribution process, encourage continuous learning, improve collaboration, and help beginner developers build practical experience and showcase their skills more effectively.